

Sistema ERP

Arquitectura, módulos funcionales y estado actual de la plataforma

Versión del documento	1.0
Fecha	Septiembre de 2026
Alcance	Backend (API) y Frontend (panel administrativo)
Repositorio	sistema-erp (monorepo pnpm)

Contenido

1. Resumen general del proyecto
2. Arquitectura de la solución
3. Stack tecnológico
4. Organización del repositorio
5. Modelo de datos
6. Módulos funcionales del backend
7. Seguridad y control de acceso
8. Aplicación frontend
9. Base de datos, migraciones y datos iniciales
10. Estado actual y trabajo pendiente
11. Consideraciones de despliegue

1. Resumen general del proyecto

El sistema es una plataforma tipo ERP orientada a la administración de operaciones comerciales de una o varias empresas dentro de un mismo grupo: gestión de usuarios y permisos, estructura organizacional (empresas, sucursales, bodegas y ubicaciones), catálogo de productos, proveedores y clientes, con un módulo de auditoría que registra la actividad relevante de cada usuario sobre el sistema.

El proyecto está organizado como un monorepo con dos aplicaciones independientes que se comunican por HTTP: un backend construido sobre NestJS que expone una API REST documentada con Swagger, y un frontend construido sobre Vue 3 que consume dicha API y ofrece un panel administrativo con control de acceso por permisos.

El diseño contempla desde el inicio operación **multiempresa**: un mismo usuario puede tener acceso a más de una empresa del grupo, y cada solicitud al backend indica sobre cuál empresa se está trabajando mediante una cabecera dedicada, validada en el servidor contra las empresas a las que el usuario tiene acceso real.

2. Arquitectura de la solución

El sistema sigue una arquitectura cliente-servidor clásica, con separación estricta entre backend y frontend:

Backend (API)

API REST construida con NestJS, organizada por módulos de dominio. Cada módulo separa la lógica de aplicación (servicios, DTOs de entrada) de la capa de presentación (controladores HTTP), siguiendo una variante simplificada de arquitectura por capas:

- `application/` — servicios de negocio y DTOs con validación (`class-validator`)
- `presentation/` — controladores, guards y decoradores expuestos al exterior

El acceso a datos se centraliza a través de Prisma ORM, inyectado como servicio (`PrismaService`) dentro de la capa de infraestructura, evitando que la lógica de negocio dependa directamente del driver de base de datos.

Frontend (panel administrativo)

Aplicación de página única (SPA) construida con Vue 3 (Composition API) y Vite. La navegación se gestiona con Vue Router, protegida por un guard global que valida sesión y permisos antes de renderizar cada vista. El estado de sesión, tema visual y notificaciones se gestionan con Pinia mediante stores independientes.

La comunicación con la API se realiza a través de una instancia de Axios centralizada que añade automáticamente el token de acceso y la empresa activa a cada petición, y que renueva la sesión de forma transparente cuando el token expira.

Ambas aplicaciones son independientes en tiempo de ejecución y se despliegan por separado; el frontend se comunica con el backend exclusivamente a través de la API pública bajo el prefijo

/api .

3. Stack tecnológico

Backend

Categoría	Tecnología	Uso en el proyecto
Framework	NestJS 11	Estructura modular de la API, inyección de dependencias, guards, pipes y filtros
Lenguaje	TypeScript	Tipado estático en toda la capa de servidor
Base de datos	PostgreSQL	Almacenamiento relacional principal
ORM	Prisma 7 (driver adapter <code>@prisma/adaptar-pg</code>)	Modelado de datos, migraciones y acceso tipado a la base de datos
Autenticación	Passport + <code>@nestjs/jwt</code>	Estrategia JWT para proteger rutas privadas
Hash de contraseñas	Argon2	Almacenamiento seguro de credenciales
Validación	class-validator / class-transformer	Validación y transformación de los DTOs de entrada
Documentación de API	Swagger (<code>@nestjs/swagger</code>)	Documentación interactiva disponible en <code>/api/docs</code>
Seguridad HTTP	Helmet, CORS con credenciales, <code>@nestjs/throttler</code>	Cabeceras de seguridad, control de orígenes y límite de peticiones
Archivos	Multer	Carga de imágenes de producto, servidas como recurso estático
Cookies	cookie-parser	Manejo del refresh token como cookie httpOnly

Frontend

Categoría	Tecnología	Uso en el proyecto
Framework	Vue 3 (Composition API, <code><script setup></code>)	Construcción de la interfaz del panel administrativo
Build tool	Vite 8	Servidor de desarrollo y empaquetado de producción
Enrutamiento	Vue Router 4	Rutas protegidas por sesión y por permiso
Estado global	Pinia	Stores de autenticación, tema y notificaciones
Estilos	Tailwind CSS 4 (plugin Vite)	Sistema de utilidades para la interfaz, con soporte de tema claro/oscuro
Peticiones HTTP	Axios	Cliente HTTP con interceptores de autenticación y reintentos
Iconografía	lucide-vue-next	Iconos utilizados en la interfaz
Lenguaje	TypeScript	Tipado estático en componentes, stores y servicios

Node.js 22

pnpm workspaces

PostgreSQL

REST + JSON

JWT (access + refresh)

RBAC granular

Swagger / OpenAPI

4. Organización del repositorio

El proyecto se maneja como un monorepo administrado con `pnpm-workspace.yaml` , con dos carpetas principales en la raíz:

Carpeta	Contenido
backend/	API NestJS, esquema y migraciones de Prisma, scripts de semilla (seed) y de verificación de módulos, uploads de archivos
frontend/	Aplicación Vue 3, componentes, vistas, stores, servicios de red y configuración de Vite/Tailwind
docs/	Documentación del proyecto (este documento)

Estructura interna del backend

Dentro de `backend/src` , cada dominio de negocio vive en su propio módulo bajo `modules/` :

auth

users

roles

permissions

organization

suppliers

products

customers

audit

dashboard

business

El código transversal (guards, decoradores, filtros de excepción, constantes de permisos, utilidades) se agrupa en `src/common`, y la configuración de acceso a datos (cliente de Prisma) en `src/infrastructure`.

Estructura interna del frontend

Dentro de `frontend/src`:

Carpeta	Contenido
<code>views/</code>	Una vista por módulo funcional (dashboard, usuarios, roles, permisos, organización, proveedores, productos, clientes, auditoría, login)
<code>layouts/</code>	Estructura general del panel administrativo (sidebar, topbar)
<code>components/base</code>	Componentes de interfaz reutilizables (botón, input, tarjeta, badge)
<code>components/feedback</code>	Centro de notificaciones de la aplicación
<code>components/forms</code>	Campos de formulario especializados (ubicación geográfica nacional)
<code>stores/</code>	Estado global: sesión (<code>auth.store</code>), tema (<code>theme.store</code>), notificaciones (<code>feedback.store</code>)
<code>services/</code>	Cliente HTTP (<code>http.service</code>) y contexto de empresa activa (<code>company-context</code>)
<code>composables/</code>	Lógica reutilizable, como la verificación de permisos (<code>usePermissions</code>)
<code>router/</code>	Definición de rutas y guard de navegación

5. Modelo de datos

El esquema de base de datos se define en Prisma (`backend/prisma/schema.prisma`) y se agrupa conceptualmente en los siguientes bloques:

Seguridad y control de acceso

`User`, `Employee`, `Role`, `Permission`, `Module`, `UserRole`, `RolePermission`, `UserCompany`, `RefreshToken`, `Log`. Modelan un esquema RBAC (control de acceso basado en roles) clásico: un usuario puede tener varios roles, un rol agrupa varios permisos, y cada permiso pertenece a un módulo del sistema.

Geografía

`Country`, `Department`, `Municipality`, `District`. División político-administrativa utilizada para ubicar empleados, sucursales y clientes; se carga mediante un script de semilla con la geografía de El Salvador (departamentos, municipios y distritos).

Organización

`Company` , `Branch` , `WarehouseCategory` , `Warehouse` , `Location` . Representan la estructura física del negocio: empresas del grupo, sus sucursales, las bodegas de cada sucursal y las ubicaciones puntuales dentro de cada bodega (usadas para el control de inventario por posición).

Catálogo

`Product` , `ProductCategory` , `ProductSubcategory` , `ProductUnit` , `ProductImage` , `ProductSupplier` . Definen el catálogo de productos, su clasificación jerárquica (categoría / subcategoría), su unidad de medida, imágenes asociadas y los proveedores habilitados para cada producto.

Terceros

`Customer` , `Supplier` , `SupplierContact` . Clientes y proveedores del negocio, con contactos asociados en el caso de proveedores.

Operación comercial

`InventoryStock` , `Purchase` / `PurchaseItem` , `Quotation` / `QuotationItem` , `Sale` / `SaleItem` , `Transfer` / `TransferItem` , `Vehicle` , `Driver` . Cubren existencias por ubicación, compras a proveedores, cotizaciones a clientes, ventas, traslados entre bodegas y datos de flota de reparto. Este bloque se detalla en la sección 10, ya que su exposición como módulos independientes con permisos propios está pendiente.

Convenciones del esquema. Todas las tablas de negocio usan `id` autoincremental, marcas de tiempo `createdAt` / `updatedAt` y borrado lógico mediante `deletedAt` en lugar de eliminación física, lo que permite conservar el historial y restaurar registros cuando es necesario. Las tablas se mapean a `snake_case` en PostgreSQL mediante `@map` , manteniendo `camelCase` en el código TypeScript.

6. Módulos funcionales del backend

Cada módulo expone su propio conjunto de rutas bajo `/api` , protegidas por autenticación JWT y, cuando aplica, por permisos específicos.

Autenticación (auth)

Inicio de sesión con usuario y contraseña, emisión de token de acceso (JWT de corta duración) y token de refresco (opaco, almacenado con hash SHA-256 en base de datos y rotado en cada uso). Incluye bloqueo temporal de la cuenta tras 5 intentos fallidos consecutivos, endpoint de sesión actual (`/auth/me`) y cierre de sesión con revocación del refresh token.

Usuarios (users)

Alta, edición, cambio de estado y asignación de roles a usuarios; cambio de contraseña. Vinculado al registro de empleado correspondiente.

Roles y permisos (roles, permissions)

Administración de roles, duplicación de roles existentes, asignación de permisos por rol y administración del catálogo de permisos agrupado por módulo. El rol `superadmin` tiene acceso total y omite la validación de permisos.

Organización (organization)

Gestión de empresas, sucursales, categorías de bodega, bodegas y ubicaciones. Base para la operación multiempresa: define qué empresas y sucursales existen y cómo se agrupan sus bodegas.

Proveedores (suppliers)

Registro de proveedores y sus contactos, con catálogos de apoyo y filtros de búsqueda paginados.

Productos (products)

Catálogo de productos con categorías, subcategorías, unidades de medida, imágenes y proveedores asociados por producto. Incluye endpoint de catálogo consolidado para consumo desde el frontend.

Clientes (customers)

Registro y administración de clientes, con cambio de estado (activo/inactivo).

Auditoría (audit)

Registro y consulta de la bitácora de acciones del sistema (quién hizo qué, cuándo y sobre qué registro), utilizada, entre otros casos, para dejar constancia de inicios de sesión fallidos y exitosos.

Panel de indicadores (dashboard)

Resumen ejecutivo con conteos generales (empresas, sucursales, bodegas, ubicaciones, clientes, proveedores, productos), existencias totales y productos con inventario bajo, totales de ventas y compras, y últimas acciones registradas en la auditoría.

7. Seguridad y control de acceso

Autenticación

- Contraseñas almacenadas con **Argon2**, nunca en texto plano ni con algoritmos reversibles.
- Token de acceso **JWT** de vida corta, enviado en la cabecera `Authorization: Bearer`.
- Token de refresco de vida más larga, entregado como cookie **httpOnly**, almacenado en base de datos como hash y **rotado** en cada renovación: el token usado se revoca de inmediato, evitando su reutilización.
- Bloqueo de cuenta tras 5 intentos fallidos consecutivos, con reinicio del contador al autenticarse correctamente.

Autorización

- Modelo RBAC: permisos con formato `recurso.acción` (por ejemplo `users.create`, `products.update`), agrupados por módulo y asignados a roles.
- Un `PermissionsGuard` global valida, por cada ruta protegida, que el usuario autenticado cuente con el o los permisos requeridos, en modo `ALL` (todos) o `ANY` (al menos uno), según se declare en el controlador.
- Soporte especial para acciones de cambio de estado (activar/desactivar), donde el permiso exigido depende de si la operación activa o desactiva el registro.
- El rol `superadmin` omite la validación de permisos y tiene acceso a todas las empresas del grupo.

Aislamiento por empresa (multiempresa)

Cada usuario tiene asociadas una o varias empresas mediante `UserCompany`. El backend resuelve la empresa activa de cada solicitud a partir de la cabecera `X-Company-Id`, validando siempre que el usuario tenga acceso real a esa empresa antes de continuar; de lo contrario responde con error de acceso prohibido. Esta validación se centraliza en `CompanyScopeService` para evitar duplicarla en cada módulo.

Otras medidas

- **Helmet** para cabeceras HTTP de seguridad y **CORS** restringido a los orígenes configurados, con soporte de credenciales.
- **Throttling** de peticiones (`@nestjs/throttler`) para mitigar abuso de la API.
- Validación estricta de entrada en todos los endpoints (`whitelist` y `forbidNonWhitelisted` activados), rechazando cualquier campo no declarado en el DTO correspondiente.
- Filtro global de excepciones que normaliza el formato de las respuestas de error.

8. Aplicación frontend

Navegación y control de acceso

Las rutas se definen en `router/index.ts` y cada vista privada declara el permiso mínimo requerido para acceder a ella (por ejemplo, la vista de usuarios exige `users.view`). Un guard global de navegación verifica, antes de cada cambio de ruta, que exista sesión activa y que el usuario tenga el permiso correspondiente; en caso contrario redirige al login o al dashboard según corresponda. Los usuarios con rol `superadmin` tienen acceso a todas las rutas.

Sesión y renovación de token

El token de acceso se mantiene únicamente en memoria (store de Pinia), no en almacenamiento persistente del navegador, reduciendo la superficie de exposición ante ataques de tipo XSS. Cuando una petición responde con 401, el interceptor de Axios intenta renovar la sesión automáticamente usando el refresh token (cookie httpOnly) y reintenta la petición original; si la renovación falla, se cierra la sesión y se redirige al login. Las peticiones GET también cuentan con reintento automático ante errores de red o de disponibilidad (502/503/504).

Interfaz

El panel administrativo usa un layout con barra lateral de navegación y barra superior, con soporte de tema claro/oscuro gestionado por un store dedicado. Los componentes de interfaz (botones, tarjetas, campos, badges) se centralizan en `components/base` para mantener consistencia visual en todas las vistas, y un centro de notificaciones (`AppNoticeCenter`) muestra los mensajes de éxito y error de forma uniforme en toda la aplicación.

Vistas implementadas

Vista	Ruta	Permiso requerido
Inicio de sesión	/login	Pública
Panel de indicadores	/dashboard	Sesión activa
Usuarios	/users	users.view
Roles	/roles	roles.view
Permisos	/permissions	permissions.view
Organización	/organization	companies.view
Proveedores	/suppliers	suppliers.view
Productos	/products	products.view
Clientes	/customers	customers.view
Auditoría	/audit	logs.view

9. Base de datos, migraciones y datos iniciales

La base de datos se administra íntegramente con Prisma Migrate. El historial de migraciones refleja la evolución del proyecto en etapas claramente diferenciadas:

Migración	Contenido
init_rbac	Modelo base de usuarios, roles, permisos y módulos
add_refresh_tokens	Soporte de sesión persistente con rotación de refresh tokens
add_organization_structure	Empresas, sucursales, bodegas y ubicaciones
add_business_modules	Entidades operativas: inventario, compras, ventas, cotizaciones, traslados, flota
complete_ers_v06	Consolidación del modelo entidad-relación del sistema
national_geography	División político-administrativa (departamentos, municipios, distritos)
fix_la_union_district	Corrección puntual de datos geográficos
product_catalog	Categorías, subcategorías, unidades e imágenes de producto
multi_company_scope	Ajustes para el aislamiento de datos por empresa

Datos iniciales (seed)

El script `prisma/seed.ts` inicializa el catálogo de módulos y permisos del sistema, crea el rol `superadmin` con acceso total y carga la división geográfica completa de El Salvador (departamentos, municipios y distritos) desde `prisma/el-salvador-geography.ts`, utilizada como base para direcciones de empleados, sucursales y clientes.

10. Estado actual y trabajo pendiente

Los módulos descritos en la sección 6 (autenticación, usuarios, roles y permisos, organización, proveedores, productos, clientes, auditoría y dashboard) están completos de extremo a extremo: modelo de datos, API con permisos propios y vista en el frontend.

El módulo `business` agrupa, además, un conjunto de endpoints operativos (inventario, compras, cotizaciones, ventas, traslados entre bodegas y datos de flota de reparto) que ya cuentan con su modelo de datos en Prisma y con lógica de servicio funcional en el backend, pero que todavía no forman parte del catálogo activo de módulos y permisos ni tienen una vista propia en el frontend. Corresponden a la siguiente etapa de desarrollo del sistema y quedan pendientes de:

- Independizarse en módulos propios (application/presentation) siguiendo el mismo patrón que productos, clientes o proveedores.
- Incorporar sus permisos al catálogo activo de permisos y asignarlos a los roles correspondientes.
- Construir las vistas de frontend correspondientes (inventario, compras, cotizaciones, ventas, traslados y flota).

Nota de alcance. Estas capacidades no deben considerarse ausentes del modelo del negocio: la base de datos y la lógica de cálculo de inventario ya existen y son funcionales a nivel de API; lo pendiente es su integración formal al esquema de seguridad y a la interfaz de usuario del panel administrativo.

11. Consideraciones de despliegue

- El backend expone su API bajo el prefijo `/api` y su documentación interactiva en `/api/docs` (Swagger); escucha en el puerto configurado mediante variables de entorno y responde en todas las interfaces de red (`0.0.0.0`).
- Los orígenes permitidos por CORS se calculan a partir de la URL configurada del frontend, más los orígenes de desarrollo local, y las peticiones se aceptan con credenciales para permitir el uso de cookies de sesión.

- Las imágenes de producto se almacenan en el sistema de archivos del servidor (carpeta `uploads/`) y se sirven como recurso estático bajo la ruta `/uploads` .
- El frontend se construye como una aplicación estática con Vite (`vue-tsc` más `vite build`) y puede servirse desde cualquier servidor de contenido estático o CDN, apuntando mediante variable de entorno a la URL pública del backend.
- La conexión a PostgreSQL se gestiona mediante un driver adapter nativo (`@prisma/adapter-pg`), y el esquema se mantiene sincronizado en cada entorno mediante `prisma migrate` .